BirdNET no ESP32-S3

Manual de teste no PC, treinamento, compilação e gravação

Projeto de identificação acústica de aves | ESP32-S3 N16R8 | Windows / PowerShell | 14 de outubro de 2026



O que já foi feito nesta placa

A imagem final foi gravada na COM46 e comparada com o conteúdo da flash. O ESP32 iniciou, reconheceu 8 MB de PSRAM e passou no autoteste do processamento de áudio e da inferência. O autoteste usa um WAV embutido no firmware.

Verificação	Resultado observado
Flash / PSRAM	16 MB / 8 MB; teste da PSRAM OK
Features do áudio embutido	0 divergências em 5.320 valores INT8
Inferência INT8	Classe correta; diferença máxima de 1 LSB
Classe do autoteste	Eurasian Blackbird / Turdus merula
Memória da arena utilizada	202.444 bytes
Microfone / cartão SD	Ainda não conectados; SD não é necessário

Este é um exemplo de bancada com 10 espécies e uma classe Background. Não inclui gralha-azul. O reconhecimento de aves locais exige dados rotulados, treinamento e avaliação de um modelo novo.

O objetivo deste manual é permitir repetir o que foi feito e criar uma nova versão sem substituir silenciosamente a versão validada. O caminho de treinamento foi testado com áudio sintético; isso valida as ferramentas, não a precisão de um classificador de aves reais.

1. Arquivos, versão gravada e mapa do projeto

A pasta principal do projeto é:

```
C:\Users\luizf\OneDrive\Documentos\MESTRADO\Tecnologias Emergentes de Comunicação para Smart Cities\Trabalhos\passaros
```

Imagem exata gravada na COM46

```
C:\Users\luizf\OneDrive\Documentos\MESTRADO\Tecnologias Emergentes de Comunicação para Smart Cities\Trabalhos\passaros\releases\20261014-COM46-validado\flash-full.bin
```

Imagem completa: 703,440 bytes. Endereço de gravação: 0x0. Ela reúne bootloader, tabela de partições e aplicação.

SHA-256 da imagem completa:

```
6aa9b27f9e9775f0204f55b21f40d01ff87dd0c8434f0adbef55c626f7091df0
```

A aplicação isolada é releases/20261014-COM46-validado/passaros_tiny.bin (637,904 bytes); seu endereço é 0x10000. Não gravar a aplicação isolada em 0x0.

Pasta / arquivo	Função
firmware/	Fontes do exemplo baseline, com correção das camadas densas
releases/20261014-COM46-validado/	Cópia da imagem, fontes, configuração, hashes e recibo
models/custom/<versão>/	Modelo treinado, labels, referência e avaliação
firmware-custom/<versão>/	Projeto C++ gerado para um modelo novo
artifacts-custom/<versão>/	Binários compilados desse projeto
scripts/	Ferramentas locais descritas neste manual
upstream/	Repositórios oficiais preservados
results/hardware/	Logs da gravação, verificação e boot

O primeiro build, que falhou na inferência embarcada, está preservado em releases/20261014-COM46-baseline para diagnóstico. A versão final é a pasta com o sufixo validado.

Backup completo anterior à primeira gravação:

```
C:\Users\luizf\OneDrive\Documentos\MESTRADO\Tecnologias Emergentes de Comunicação para Smart Cities\Trabalhos\passaros\backups\COM46-20261014-antes.bin
```

2. Entender o caminho e preparar o ambiente

Modelo e firmware são coisas diferentes: o modelo contém pesos aprendidos e a ordem das classes; o firmware contém captura I2S, processamento do som, execução do modelo e saída serial. Para trocar as aves reconhecidas é necessário trocar o modelo, os labels e os vetores do autoteste, e recompilar.

O BirdNET-Analyzer é a ferramenta de desktop para analisar gravações. O caminho embarcado usa o processamento do projeto BirdNET/BioDCASE-Tiny e TensorFlow Lite Micro. O script local de treinamento aprende uma CNN pequena do zero; não transfere pesos nem destila o modelo grande do BirdNET.

Ferramentas deste PC

Ferramenta	Uso
Python 3.11 / .venv	Extração de features, treinamento e geração
TensorFlow 2.21 / LiteRT 2.3	Treinar, converter e executar INT8 no host
ESP-IDF 5.5.2	Compilar para ESP32-S3
esptool 5.5 / pyserial	Gravar, verificar e ler a COM46
GCC/G++	Comparar o DSP C++ com a referência Python
FFmpeg	Converter e recortar gravações
ReportLab / Poppler	Gerar e revisar este PDF

Abra o PowerShell na pasta principal do projeto. Os comandos seguintes partem dessa pasta. A instalação está pronta. Se precisar reinstalar o ambiente Python nesta mesma cópia:

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\setup.ps1
uv pip check --python .venv/Scripts/python.exe
```

setup.ps1 instala as versões do lock e prepara o exemplo baseline. Ele não instala o ESP-IDF do zero: usa a instalação existente em C:\Espressif\frameworks\esp-idf-v5.5.2. Em outro computador será necessário instalar esse SDK e adaptar o caminho no launcher.

O projeto não inicia servidor, banco ou container e não precisa de Wi-Fi para inferir. O cartão SD não participa deste exemplo; modelo na flash, amostras na RAM e logs pela UART.

2A. Escutar pelo microfone do PC

É possível testar o modelo Tiny diretamente no computador, sem conectar o ESP32 ou seu microfone I2S. Abra o PowerShell na pasta principal deste projeto e execute:

```
.\.venv\Scripts\python.exe scripts\listen_tiny.py
```

Espere a mensagem Escutando blocos de 3 segundos. O programa abre o microfone padrão do Windows, analisa blocos de 3 segundos e imprime uma classe por bloco. A primeira inicialização pode demorar mais para carregar o processamento. Ctrl+C encerra e libera o microfone.

Experimentar com um canto do YouTube

Procure Eurasian Blackbird song ou Turdus merula canto: é o melro do áudio de exemplo. Prefira um canto isolado, sem música ou narração. Dê play pelos alto-falantes para que o microfone do PC capte o som. Com fones de ouvido, o microfone pode não ouvir a reprodução. Ajuste o volume para captar o canto sem distorção.

O script lê o microfone; não captura diretamente a faixa de áudio do navegador. A acústica da sala, distância, vento, fala e volume podem mudar a classificação. O resultado esperado para um canto reconhecido do melro é classe=Eurasian Blackbird, mas isso não é garantido para qualquer vídeo.

Escolher outro microfone

```
.\.venv\Scripts\python.exe scripts\listen_tiny.py --list-devices
.\.venv\Scripts\python.exe scripts\listen_tiny.py --device 9
```

9 é apenas um exemplo: use o ID de uma entrada com canais in na listagem atual. Os IDs podem mudar. Sem --device, o programa usa a entrada padrão. Para um teste limitado a dois blocos, cerca de 6 segundos de áudio:

```
.\.venv\Scripts\python.exe scripts\listen_tiny.py --windows 2
```

Dependência e problemas de captura

sounddevice 0.5.6 já está instalado e registrado no requirements.lock.txt. Se faltar ao reutilizar uma .venv antiga, instale somente essa dependência:

```
uv pip install --python .venv/Scripts/python.exe sounddevice==0.5.6
```

Sem entrada ou sem sinal: confira o microfone selecionado, mute, volume e a permissão de microfone para aplicativos de área de trabalho nas configurações do Windows. Audio quase nulo indica RMS menor que 8; esse bloco não é classificado. Se houver perda de amostras, o programa avisa e descarta o bloco.

2B. Entender o teste no PC e seus resultados

O PC usa os mesmos pesos do ESP32?

Sim, neste teste Tiny. listen_tiny.py e infer_tiny.py carregam diretamente firmware/main/model.tflite e a ordem das classes de models/labels.json. Os pesos são os valores aprendidos pela rede durante o treinamento e estão dentro do arquivo .tflite. Escutar ou analisar um WAV executa o modelo sem treinar nem alterar esses pesos.

Os hashes SHA-256 de models/biodcase_baseline.tflite, firmware/main/model.tflite e do modelo preservado em releases/20261014-COM46-validado/source/main/model.tflite coincidiram. O PC usa LiteRT; a placa usa TensorFlow Lite Micro. Os kernels podem gerar pequenas diferenças numéricas: o autoteste registrado da placa manteve a classe e teve erro máximo de 1 unidade INT8.

Do som até a classe

No PC: microfone -> bloco de 3 segundos -> conversão para mono PCM16 a 24 kHz -> features [1,1,40,133] -> modelo INT8 -> classe de maior pontuação. O script captura na taxa padrão do dispositivo e reamostra quando necessário; neste PC foi observada captura a 44.100 Hz. No ESP32, a origem é o microfone I2S e o DSP é executado em C++.

Ler a saída do listen

```
classe=Eurasian Blackbird score_relativo=0.854
  rms=850.4 pico=4200 clipping=0
```

Exemplo ilustrativo, dividido em duas linhas para caber na página; o programa imprime em uma linha. classe é o rótulo vencedor; score_relativo é a softmax entre as classes e não uma probabilidade calibrada de acerto; rms indica nível médio; pico é a maior amplitude; clipping conta amostras saturadas. Há 10 espécies e Background. Sons de aves desconhecidas podem receber uma classe conhecida.

Testar arquivos e consultar o CSV

```
.\.venv\Scripts\python.exe scripts\infer_tiny.py
.\.venv\Scripts\python.exe scripts\infer_tiny.py `
  --audio data\meu_audio.wav
```

Sem --audio, roda os 11 WAVs de exemplo: o teste registrado acertou 9, o que verifica a execução e não estima precisão em campo. O WAV próprio deve ter exatamente 3 segundos, mono, 24 kHz e PCM16. A saída padrão é results/tiny_python.csv, com resumo em results/tiny_python.json; outra execução substitui esses dois relatórios. Use --output results/meu_teste.csv para separá-los.

listen_tiny.py mostra resultados no terminal e não salva WAV ou CSV. scripts/analyze.ps1 usa outro modelo: BirdNET 2.4 completo no PC, com saída em results/analyzer/BirdNET_CombinedTable.csv. Esse CSV não é o resultado do Tiny embarcado.

3. Definir classes e organizar gravações

Comece definindo uma lista pequena de aves de interesse e uma classe Background. Background deve incluir ruídos relevantes do local e sons que não sejam das espécies-alvo. Acrescentar o nome de uma espécie a labels.json não ensina seu canto ao modelo.

Gralha-azul pode ser uma classe Cyanocorax_caeruleus, mas só será treinada depois que houver gravações verificadas dessa espécie. Revisar a identidade da ave, a qualidade da gravação e as condições de uso de cada fonte. Guardar procedência, local, data, autoria e licença.

Estrutura exigida por train_tiny.py

```
data/dataset/
  metadata.csv
  Train/
    Background/*.wav
    Cyanocorax_caeruleus/*.wav
  Validation/
    Background/*.wav
    Cyanocorax_caeruleus/*.wav
  Test/
    Background/*.wav
    Cyanocorax_caeruleus/*.wav
```

A pasta define a classe: um WAV em Cyanocorax_caeruleus recebe esse rótulo, qualquer que seja seu nome. Train ajusta pesos, Validation orienta a escolha do modelo e Test mede o resultado final. Todas as classes devem ter WAVs nos três conjuntos.

Divida por gravação original antes de cortar os trechos. Janelas da mesma gravação, indivíduo ou sessão não devem cair em conjuntos diferentes. Separar também locais/datas quando possível torna a avaliação mais próxima do uso em campo.

Arquivo metadata.csv obrigatório

```
path,recording_id
Train/Background/bg001_000.wav,bg001
Train/Background/bg001_001.wav,bg001
Validation/Background/bg002_000.wav,bg002
Test/Background/bg003_000.wav,bg003
```

Liste todos os WAV de todas as classes. path é relativo a data/dataset e usa barras /. recording_id identifica a gravação original, repetindo o mesmo ID para seus recortes. O script bloqueia IDs ou PCM idênticos em conjuntos diferentes e arquivos ausentes/duplicados. Isso não substitui a revisão da origem dos dados.

3A. Como ensinar o canto de uma nova espécie

O fluxo de treinamento já está implementado em scripts/train_tiny.py; as etapas detalhadas estão nas seções 3 a 8. Para ensinar gralha-azul, por exemplo, é preciso reunir áudios reais dessa espécie e ruídos do ambiente. Reproduzir um vídeo no listen apenas testa o modelo atual; não ensina um canto novo.

Classes e arquivos

1. Escolha as espécies-alvo. Para reconhecer uma espécie independentemente de seu tipo de vocalização, coloque exemplos revisados dos cantos e chamados desejados na mesma pasta da espécie. Este script usa uma classe por arquivo; evite trechos com identificação ambígua ou várias espécies-alvo simultâneas.

2. Reúna gravações independentes, com variação de indivíduos, distâncias e ambientes. Inclua Background com ruídos e sons fora das espécies-alvo. Um único vídeo recortado em muitas janelas não equivale a muitas gravações independentes. A quantidade necessária depende da diversidade e da avaliação; não há mínimo de áudios validado neste projeto.

3. Separe as gravações originais entre Train, Validation e Test antes dos cortes. Depois extraia os trechos de 3 segundos no formato da seção 4 e revise seus rótulos. Coloque os WAVs nas pastas de cada classe da seção 3.

Preencher data/dataset/metadata.csv

```
path,recording_id
Train/Cyanocorax_caeruleus/g001_000.wav,g001
Train/Cyanocorax_caeruleus/g001_001.wav,g001
Validation/Cyanocorax_caeruleus/g002_000.wav,g002
Test/Cyanocorax_caeruleus/g003_000.wav,g003
```

4. Liste todos os WAVs, inclusive Background. O path é relativo a data/dataset, usando /. recording_id identifica a gravação original; repita o mesmo ID em seus recortes e mantenha todos no mesmo conjunto. Esses nomes são exemplos: g001 e bg001 não são áudios reais já fornecidos. O CSV registra a origem; a classe vem da pasta.

Treinar, avaliar e levar para a placa

5. Execute o comando da seção 5 em uma pasta de saída nova. Consulte evaluation.json e compare FP32 e INT8 conforme a seção 6. Quando o resultado for adequado, gere firmware-custom com prepare_custom.py, compile e valide na placa conforme as seções 7 a 11.

O treino local cria uma CNN do zero com as classes do seu dataset. Background + gralha-azul produz um modelo dessas duas classes; não acrescenta automaticamente gralha às dez espécies do baseline. Para manter outras espécies, inclua seus dados no novo treino. Os scripts listen_tiny.py e infer_tiny.py atuais continuam apontando para o baseline e não selecionam um bundle customizado automaticamente. O treino avalia o novo modelo no PC e prepare_custom.py executa sua referência antes da compilação.

4. Preparar WAV e manter o mesmo processamento

Cada entrada de treinamento deve ter exatamente 3 segundos, um canal, 24.000 amostras por segundo e PCM de 16 bits: 72.000 amostras por arquivo. O script rejeita duração, número de canais ou taxa diferentes.

Converter um trecho de 3 segundos

```
ffmpeg -n -ss 12 -i data\original.m4a -t 3 `
  -ac 1 -ar 24000 -c:a pcm_s16le data\trecho.wav
```

O exemplo começa no segundo 12. -n impede sobrescrever um arquivo existente. A gravação precisa ter duração suficiente para produzir os 3 segundos completos.

Cortar vários trechos após escolher o conjunto

```
ffmpeg -n -i data\original.wav -ac 1 -ar 24000 `
  -c:a pcm_s16le -f segment -segment_time 3 `
  data\dataset\Train\Cyanocorax_caeruleus\g001_%03d.wav
```

Criar a pasta antes. O último trecho pode ter menos de 3 segundos: não o inclua no dataset nem no metadata.csv. Preserve o original e revise quais trechos realmente contêm a classe rotulada. Não confundir uma pasta de arquivos com rótulos confiáveis.

Contrato DSP preservado

Parâmetro	Valor
Áudio	3 s / 24 kHz / mono / int16
Janela / passo	4.096 / 512 amostras
Features	40 bandas, 133 janelas
Faixa mel	125 a 7.500 Hz
Normalização	Min-max por trecho; silêncio tem tratamento explícito
Entrada INT8	[1, 1, 40, 133]

As features resumem como a energia do som se distribui ao longo do tempo e das frequências. O modelo aprende padrões nessa matriz. Se o PC e o ESP32 produzirem matrizes diferentes para o mesmo som, o comportamento aprendido não se transfere corretamente.

Não mudar sample rate, FFT, mel, layout, normalização ou quantização só no Python. Qualquer mudança desse contrato exige adaptar o DSP C++ e repetir a comparação de referência.

5. Treinar uma nova versão

Com o dataset e metadata.csv revisados, execute o treinamento. Use um nome de saída novo para cada experimento; o script recusa uma pasta já existente.

```
.\.venv\Scripts\python.exe scripts\train_tiny.py `
  --dataset data\dataset --output models\custom\gralha-v1 `
  --epochs 30 --batch-size 16 --seed 42
```

30 épocas é uma configuração inicial do comando, não uma garantia de convergência ou qualidade. O script monitora val_loss, usa EarlyStopping com paciência de 5 épocas e restaura os melhores pesos. O treinamento usa CPU no Windows nativo; este fluxo não configura CUDA/WSL.

O que o script faz

1. Verifica gravações, classes, split por origem e duplicatas. 2. Extrai as mesmas features da referência BioDCASE. 3. Treina uma CNN pequena com convoluções, pooling e camadas densas. 4. Ajusta o peso das classes conforme a quantidade de exemplos. 5. Salva o modelo e converte para INT8. 6. Avalia FP32 e INT8 no conjunto Test.

A rede recebe a matriz [1, 40, 133], rearranja para o layout usado nas convoluções e devolve logits, um número por classe. Os labels são ordenados e salvos junto do modelo. A posição de cada label precisa permanecer ligada à posição da saída.

Teste das ferramentas sem dataset real

```
.\.venv\Scripts\python.exe scripts\train_tiny.py `
  --smoke --epochs 1 --output models\custom\smoke-novo
```

Este comando gera áudio sintético com duas classes: Background e Demo_chirp. Foi usado para validar o encadeamento treinamento -> conversão -> exportação -> compilação. Não é um modelo treinado para aves e não deve substituir o firmware de identificação em campo.

Interpretar o treinamento

Loss mede o erro usado para ajustar os pesos. Accuracy mede a fração de acertos naquele conjunto. Se o erro de Train cai e Validation piora, há sinal de overfitting. Para comparar configurações use Validation; só consulte Test após fixar a escolha.

6. Conversão INT8 e avaliação

A quantização representa pesos e ativações com inteiros menores, reduzindo armazenamento e custo de processamento. O treino acontece em ponto flutuante; a etapa INT8 aproxima os valores usando scale e zero_point.

```
q = clip(x / scale + zero_point, -128, 127)
x_aproximado = (q - zero_point) * scale
```

Neste projeto, a conversão da entrada para int8 preserva o truncamento da referência BioDCASE. Não trocar silenciosamente por outra regra de arredondamento. O conversor usa até 200 exemplos de Train para calibração, exige operações INT8 e entrada/saída INT8. Ele não usa Test para calibrar.

Arquivo no bundle	Conteúdo
model.keras	Rede e pesos em ponto flutuante
model.int8.tflite	Modelo convertido para embarcar
labels.json	Ordem das classes
metadata.json	DSP, quantização, operadores, hash e seed
reference.wav	Áudio de Validation usado no autoteste
evaluation.json	Métricas FP32/INT8, matriz de confusão e inventário

Consultar o resultado

```
.\.venv\Scripts\python.exe -c "import json; `
  from pathlib import Path; `
  p=Path('models/custom/gralha-v1/evaluation.json'); `
  print(json.loads(p.read_text())['test_int8'])"
```

A matriz de confusão mostra quais classes se confundem. Precision indica a proporção de avisos de uma classe que estavam corretos; recall indica a proporção dos exemplos dessa classe encontrados. Inspecione também falsos positivos em ruído e aves fora da lista. Defina critérios de aprovação antes de coletar os números.

Compare FP32 e INT8. Se a conversão reduzir muito as métricas, revise a calibração, os dados e a arquitetura. Um arquivo pequeno que compila não prova qualidade acústica, e a softmax dos logits não é uma confiança calibrada.

Compatibilidade das camadas densas: o treinamento local desabilita quantização por canal nessas camadas. O firmware também usa o kernel de referência TFLM para suportar o modelo oficial já quantizado por canal. As convoluções mantêm kernels ESP-NN.

7. Gerar o projeto C++ do novo modelo

Não substituir apenas model.tflite. Labels, PCM de referência, entrada INT8 e saída esperada devem ser gerados junto dele. prepare_custom.py cria uma pasta de firmware separada e preserva o baseline.

```
.\.venv\Scripts\python.exe scripts\prepare_custom.py `
  --bundle models\custom\gralha-v1 `
  --output-firmware firmware-custom\gralha-v1
```

O script confere hash, DSP, tipo/layout da entrada, quantidade de classes, logits e operadores. Executa o novo modelo no PC e compara o DSP C++ com Python para o WAV de referência. Se houver incompatibilidade, interrompe a exportação.

Arquivos gerados em firmware-custom/gralha-v1/main

Arquivo	Função
model.tflite	Bytes do novo modelo
labels.h	Nomes na ordem da saída
reference_audio.pcm	3 segundos usados no autoteste
reference_input.bin	Features INT8 esperadas
reference_output.bin	Saída INT8 esperada no host
feature_constants.h	Constantes do DSP mantido
main.cpp	Captura e inferência com número de classes adaptado
fully_connected_reference.cpp	Kernel denso com suporte por canal

training_bundle.json registra os metadados usados. O projeto usa a mesma arena de 512 KiB; se um modelo novo exigir mais, revisar o orçamento de RAM antes de aumentar. O teste na placa deve confirmar a alocação e os tempos.

Problemas que bloqueiam exportação

Hash mudou; labels não correspondem à saída; input shape diferente de [1,1,40,133]; modelo não INT8; operador não suportado; contrato DSP modificado; faixa de quantização inesperada; referência inválida; comparação C++/Python falhou. Corrija a causa e crie uma saída nova.

> Não editar o autoteste para aceitar uma classe errada. Ele existe para detectar diferenças de execução antes de analisar o microfone. Avaliação do dataset e autoteste embarcado verificam problemas diferentes.

8. Compilar para a N16R8

Compile o projeto gerado com um BuildName exclusivo. Assim o ESP-IDF usa um diretório curto separado e não mistura o build do baseline com o do modelo novo.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File scripts\build.ps1 -Action build `
  -FirmwareDir firmware-custom\gralha-v1 `
  -BuildName gralha-v1 -ArtifactsDir artifacts-custom\gralha-v1
```

Neste exemplo, o build fica em C:\Users\luizf\birdnet-build\gralha-v1. As fontes permanecem na pasta do projeto. O launcher reutiliza o ESP-IDF 5.5.2 e exporta os binários ao diretório indicado.

Saída	Uso
flash-full.bin	Imagem completa, gravação a partir de 0x0
passaros_tiny.bin	Aplicação isolada, endereço 0x10000
bootloader.bin	Inicialização, endereço 0x0
partition-table.bin	Mapa da flash, endereço 0x8000
passaros_tiny.elf	Símbolos para diagnóstico
sdkconfig / dependencies.lock	Configuração e dependências efetivas
flasher_args.json / flash_args	Endereços e argumentos da imagem

Configuração de hardware

ESP32-S3; flash quad DIO de 16 MB a 80 MHz; PSRAM octal de 8 MB a 80 MHz; CPU de 240 MHz; console UART0 a 115200 baud. Essa combinação foi conferida na COM46. Para outra placa, verificar a variante real antes de gravar.

Para trocar GPIOs ou canal do microfone, use -Action menuconfig com os mesmos FirmwareDir e BuildName, no menu Passaros - microfone I2S externo. Depois repita -Action build. As escolhas ficam no sdkconfig do diretório de build.

A mensagem Project build complete prova que a compilação terminou. Ela não grava a placa. Depois confira o tamanho, os hashes, o autoteste embarcado e a avaliação acústica.

9. Backup, gravação e verificação

A placa deste projeto está em COM46, identificada como USB-Enhanced-SERIAL CH343. Só um programa pode abrir a COM por vez. Feche monitores seriais antes de usar esptool. O exemplo usa o conector USB-UART, não depende de USB nativo.

Confirmar placa e salvar backup

```
.\.venv\Scripts\python.exe -m esptool `
  --chip esp32s3 --port COM46 flash-id
.\.venv\Scripts\python.exe -m esptool `
  --chip esp32s3 --port COM46 --baud 921600 `
  read-flash 0 ALL backups\COM46-antes-gralha-v1.bin
```

Use nome de backup novo. Nesta preparação foi salvo o conteúdo inteiro de 16 MB antes da primeira escrita. A imagem completa escreve também os intervalos preenchidos entre as regiões e pode substituir dados nesses endereços; o backup permite preservar o estado anterior.

Gravar a nova imagem e compará-la

```
.\.venv\Scripts\python.exe -m esptool `
  --chip esp32s3 --port COM46 --baud 921600 `
  write-flash --flash-mode dio --flash-size 16MB `
  --flash-freq 80m 0x0 artifacts-custom\gralha-v1\flash-full.bin

.\.venv\Scripts\python.exe -m esptool `
  --chip esp32s3 --port COM46 --baud 921600 `
  verify-flash 0x0 artifacts-custom\gralha-v1\flash-full.bin
```

write-flash deve concluir a escrita e verificar o hash. verify-flash compara novamente a imagem com a placa e deve retornar Verification successful. Se houver timeout, revisar cabo, COM e BOOT/RESET, ou reduzir o baudrate a 460800.

Ler o boot por um tempo limitado

```
.\.venv\Scripts\python.exe scripts\read_serial.py `
  --port COM46 --reset --seconds 30 `
  --output results\hardware\gralha-v1-boot.log
```

O script abre a serial, faz reset e salva o boot. Não roda para sempre. Para recuperar o estado anterior, use write-flash 0x0 com o backup completo correspondente, mantendo a placa correta. Não use um backup de outra placa.

10. Testes que funcionam só com o ESP32

Neste momento não há microfone nem cartão SD. Isso não impede testar flash, PSRAM, processamento e modelo. O arquivo reference_audio.pcm vem embutido na aplicação e reproduz a entrada do autoteste.

Sequência esperada no boot

```
Found 8MB PSRAM device
SPI SRAM memory test OK
Arena utilizada=202444
SELFTEST_FEATURES: divergencias=0/5320
SELFTEST_MODEL: erro_max_LSB=1
  previsto=Eurasian Blackbird esperado=Eurasian Blackbird
SELFTEST_OK: iniciando captura autonoma I2S GPIO4/5/6
```

Esses resultados foram observados na COM46 com a imagem final. Para um novo modelo, a classe, a arena e os tempos podem mudar, mas o teste precisa continuar comparando features e saída com a referência correspondente.

O firmware aceita erro máximo de 2 LSB na saída INT8, desde que a classe seja a mesma. LSB é uma unidade do inteiro quantizado. Ele exige comparação exata das features. Se falhar, a inicialização da captura é bloqueada.

Correção necessária encontrada na bancada

A primeira imagem tinha features corretas, mas devolveu Background para o exemplo Eurasian Blackbird. A leitura do kernel mostrou que a implementação ESP-NN das camadas densas desse commit não aplicava os multiplicadores por canal exigidos pelo modelo. A versão final registra nessas camadas o kernel de referência TFLM, que aplica a quantização por canal; as convoluções mantêm ESP-NN.

Depois da correção, a classe coincidiu com a referência e o erro máximo caiu de 156 para 1 unidade INT8. Não foi necessário mudar os pesos nem relaxar o autoteste.

> Sem microfone, Audio quase nulo rms=0.0 é esperado. Não é evidência de falha do modelo. O teste de captura acústica só poderá ser aprovado depois da ligação do microfone.

11. Quando o microfone chegar

O exemplo está configurado para um microfone digital INMP441 externo. Desligue a alimentação antes de fazer as conexões. Não é necessário cartão SD.

INMP441	ESP32-S3
VDD / VCC	3V3
GND	GND
SCK / BCLK	GPIO4
WS / LRCLK	GPIO5
SD / DOUT	GPIO6
L/R	GND, canal esquerdo

O INMP441 fornece I2S de 24 bits; o firmware usa slots de 32 bits e desloca 16 bits para formar PCM16. Se outro microfone for usado, conferir formato, alimentação, canal e shift. Não ligar VCC do INMP441 ao 5V.

Teste de áudio real

1. Ligue a placa e confirme SELFTEST_OK. 2. Fale ou reproduza um WAV conhecido próximo do microfone. 3. Confira RMS, pico e clipping. 4. Salve um WAV real e ouça a gravação. 5. Teste as aves-alvo e ruídos fora da lista. 6. Meça tempo de DSP, tempo de inferência e consumo com a alimentação prevista.

```
.\.venv\Scripts\python.exe scripts\capture_serial.py `
  --port COM46 --output data\microfone.wav
```

Feche o monitor antes da captura. Esse script envia o comando binário de exportação de 3 segundos e salva o WAV. A placa fica pausada após a exportação; reabra o monitor e envie m, ou reinicie. No monitor: p pausa, m retoma, t repete autoteste. Não envie a em um monitor textual porque a saída é PCM binário.

O ciclo atual captura 3 segundos, depois processa e infere. Há uma lacuna durante o processamento: não é captura contínua com buffer duplo. A inferência é autônoma, sem PC ou rede, mas os resultados deste exemplo só aparecem na serial; não há armazenamento persistente de eventos.

Se RMS ficar zerado com o microfone instalado, conferir 3V3, GND, SCK, WS, SD e L/R. Se houver clipping, revisar intensidade e shift. Uma espécie fora dos labels pode ser atribuída à classe mais parecida; não interpretar isso como identificação válida.

12. Repetição, diagnóstico e fontes

Sequência para cada versão nova

Dados rotulados e separados por origem -> WAV padronizados -> treinamento -> avaliação FP32/INT8 -> exportação com referência -> compilação em build separado -> backup -> gravação/verificação -> autoteste no ESP32 -> teste acústico e energético. Guarde dados, configuração, avaliação, bundle, fontes, ELF, binários, hashes e logs.

Falhas comuns

Sintoma	Próxima verificação
Pasta de saída já existe	Escolher versão nova; preservar a anterior
Vazamento de recording_id	Refazer a divisão por gravação original
WAV com tamanho incorreto	Taxa, canais, 3 s exatos e último segmento
Modelo/operador incompatível	Metadata, input shape, INT8 e resolver
Erro de caminho com acento	Usar launcher e diretório de build curto
PSRAM não encontrada	Variante N16R8, modo octal e alimentação
SELFTEST_FEATURES falhou	Contrato DSP, quantização e referência
SELFTEST_MODEL falhou	Modelo, kernels, labels e referência
COM46 ocupada	Fechar o monitor que abriu essa COM
Áudio zerado sem microfone	Resultado esperado; instalar microfone depois

Fontes primárias e arquivos locais

BioDCASE-Tiny-2026 - modelo e referência DSP

BirdNET-Analyzer - análise de desktop

BirdNET-Tiny Forge - rota oficial alternativa

Google LiteRT - quantização inteira

Espressif - TensorFlow Lite Micro

python-sounddevice - captura pelo microfone do PC

TDK - datasheet INMP441

Origem do baseline: BioDCASE commit a62675704019fc8a0bcc4e2f64346698ea501a32. Runtime ESP-TFLM: 416d3713e9312025df434d6f33600ec490705b3d. O modelo baseline segue os labels oficiais; a CNN do script local é uma adaptação para treinamento, não um novo modelo oficial BirdNET.

Projeto BirdNet - PPGCIS.pdf, páginas 1-2: ESP32-S3, microfone I2S, estudo no desktop e TinyML. Recibo desta bancada: results/hardware/receipt.json. Logs: flash-final.log, verify-final.log e serial-final.log. Licenças: NOTICE.md e firmware/LICENSE.